

CM3020 Part B – Combined Source Code

Highlighting key: Yellow shading indicates lines that differ from the provided starter code (inserted or modified). Unshaded lines match the starter code. For files not present in the starter code, the entire file is shaded.

File inventory (combined):

File	Status vs starter	Highlighted lines	Total lines
src/creature.py	Modified	80	188
src/cw-envt.py	Unchanged (starter)	0	98
src/demo_gui_elite.py	New (not in starter)	62	62
src/genome.py	Modified	33	360
src/motor_test.py	Unchanged (starter)	0	57
src/offline_from_csv.py	Unchanged (starter)	0	72
src/population.py	Unchanged (starter)	0	27
src/prepare_shapes.py	Unchanged (starter)	0	344
src/realtime_from_csv.py	Modified	136	152
src/run_ga_baseline.py	New (not in starter)	200	200
src/simulation.py	Unchanged (starter)	0	102
src/simulation_mountain.py	New (not in starter)	233	233
src/starter.py	Modified	32	37
src/test_creature.py	Unchanged (starter)	0	58
src/test_ga.py	Unchanged (starter)	0	60
src/test_ga_no_threads.py	Unchanged (starter)	0	65
src/test_genome.py	Unchanged (starter)	0	169
src/test_population.py	Unchanged (starter)	0	22
src/test_simulation.py	Unchanged (starter)	0	53

src/creature.py — Modified

Yellow shading indicates differences relative to the starter code.

```
import genome
from xml.dom.minidom import getDOMImplementation
from enum import Enum
import numpy as np

class MotorType(Enum):
    PULSE = 1
    SINE = 2

class Motor:
    def __init__(self, control_waveform, control_amp, control_freq):
        if control_waveform <= 0.5:
            self.motor_type = MotorType.PULSE
        else:
            self.motor_type = MotorType.SINE
        self.amp = control_amp
        self.freq = control_freq
        self.phase = 0

    def get_output(self):
        self.phase = (self.phase + self.freq) % (np.pi * 2)
        if self.motor_type == MotorType.PULSE:
            if self.phase < np.pi:
                output = 1
            else:
                output = -1

        if self.motor_type == MotorType.SINE:
            output = np.sin(self.phase)

        return output

class Creature:
    def __init__(self, gene_count):
        self.spec = genome.Genome.get_gene_spec()
        self.dna = genome.Genome.get_random_genome(len(self.spec), gene_count)
        self.flat_links = None
        self.exp_links = None
        self.motors = None
        self.start_position = None
        self.last_position = None

    def get_flat_links(self):
        if self.flat_links == None:
            gdicts = genome.Genome.get_genome_dicts(self.dna, self.spec)
            self.flat_links = genome.Genome.genome_to_links(gdicts)
```

```

return self.flat_links

def get_expanded_links(self):
    self.get_flat_links()
    if self.exp_links is not None:
        return self.exp_links

    exp_links = [self.flat_links[0]]
    genome.Genome.expandLinks(self.flat_links[0],
                              self.flat_links[0].name,
                              self.flat_links,
                              exp_links)
    self.exp_links = exp_links
    return self.exp_links

def to_xml(self):
    self.get_expanded_links()
    domimpl = getDOMImplementation()
    adom = domimpl.createDocument(None, "start", None)
    robot_tag = adom.createElement("robot")
    for link in self.exp_links:
        robot_tag.appendChild(link.to_link_element(adom))
    first = True
    for link in self.exp_links:
        if first:# skip the root node!
            first = False
            continue
        robot_tag.appendChild(link.to_joint_element(adom))
    robot_tag.setAttribute("name", "pepe") # choose a name!
    return '<?xml version="1.0"?>' + robot_tag.toprettyxml()

def get_motors(self):
    self.get_expanded_links()
    if self.motors == None:
        motors = []
        for i in range(1, len(self.exp_links)):
            l = self.exp_links[i]
            m = Motor(l.control_waveform, l.control_amp, l.control_freq)
            motors.append(m)
        self.motors = motors
    return self.motors

# ----- Mountain / evaluation helpers -----

def reset_episode_stats(self):
    """Reset per-run metrics (call automatically on first position update)."""
    self.max_z = None
    self.best_peak_dist = None
    self.air_steps = 0
    self.supported_steps = 0
    self.total_steps = 0
    self.peak_pos = None

```

```

def get_max_height(self):
    return 0.0 if self.max_z is None else float(self.max_z)

def get_best_peak_distance(self):
    # If never computed, return +inf so fitness becomes minimal.
    return float("inf") if self.best_peak_dist is None else float(self.best_peak_dist)

def get_air_ratio(self):
    if self.total_steps <= 0:
        return 0.0
    return float(self.air_steps) / float(self.total_steps)

def get_supported_ratio(self):
    if self.total_steps <= 0:
        return 0.0
    return float(self.supported_steps) / float(self.total_steps)

def get_mountain_fitness(self):
    """
    Fitness for Part B:
    - Reward closeness to peak (smaller distance -> higher score)
    - Penalize being airborne ("cheating") via supported_ratio
    """
    d = self.get_best_peak_distance()
    closeness = 1.0 / (1.0 + d) # in (0, 1]
    # If we never provided support info, default to no penalty.
    support = self.get_supported_ratio() if (self.air_steps + self.supported_steps) > 0 else 1.0
    return closeness * support

def update_position(self, pos, supported=None, peak_pos=None):
    """
    Update creature position and (optionally) per-step mountain metrics.
    - supported: bool (True if touching floor/mountain; False if airborne)
    - peak_pos: (x, y, z) position of mountain peak for distance-to-peak tracking
    """
    # First-ever position update for this creature in this run
    if self.start_position is None:
        self.start_position = pos
        self.last_position = pos
        self.reset_episode_stats()
    else:
        self.last_position = pos

    # Track peak position if provided
    if peak_pos is not None:
        self._peak_pos = peak_pos

    # Update per-step metrics
    self.total_steps += 1

```

```

# Max height reached
z = pos[2]
if self.max_z is None or z > self.max_z:
    self.max_z = z

# Closest distance to peak (if peak known)
if self._peak_pos is not None:
    p = np.asarray(pos, dtype=float)
    peak = np.asarray(self._peak_pos, dtype=float)
    dist = float(np.linalg.norm(p - peak))
    if self.best_peak_dist is None or dist < self.best_peak_dist:
        self.best_peak_dist = dist

# Air vs supported tracking (if provided)
if supported is not None:
    if supported:
        self.supported_steps += 1
    else:
        self.air_steps += 1

def get_distance_travelled(self):
    if self.start_position is None or self.last_position is None:
        return 0
    p1 = np.asarray(self.start_position)
    p2 = np.asarray(self.last_position)
    dist = np.linalg.norm(p1-p2)
    return dist

def update_dna(self, dna):
    self.dna = dna
    self.flat_links = None
    self.exp_links = None
    self.motors = None
    self.start_position = None
    self.last_position = None
    self.reset_episode_stats()

```

src/cw-envt.py — Unchanged (starter)

Yellow shading indicates differences relative to the starter code.

```
import pybullet as p
import pybullet_data
import time
import numpy as np
import random
import creature

p.connect(p.GUI)
p.setAdditionalSearchPath(pybullet_data.getDataPath())

import random
import pybullet as p
import math

import pybullet as p

def make_mountain(num_rocks=100, max_size=0.25, arena_size=10, mountain_height=5):
    def gaussian(x, y, sigma=arena_size/4):
        """Return the height of the mountain at position (x, y) using a Gaussian function."""
        return mountain_height * math.exp(-((x**2 + y**2) / (2 * sigma**2)))

    for _ in range(num_rocks):
        x = random.uniform(-1 * arena_size/2, arena_size/2)
        y = random.uniform(-1 * arena_size/2, arena_size/2)
        z = gaussian(x, y) # Height determined by the Gaussian function

        # Adjust the size of the rocks based on height. Higher rocks (closer to the peak) will be smaller.
        size_factor = 1 - (z / mountain_height)
        size = random.uniform(0.1, max_size) * size_factor

        orientation = p.getQuaternionFromEuler([random.uniform(0, 3.14), random.uniform(0, 3.14), random.uniform(0, 3.14)])
        rock_shape = p.createCollisionShape(p.GEOM_BOX, halfExtents=[size, size, size])
        rock_visual = p.createVisualShape(p.GEOM_BOX, halfExtents=[size, size, size], rgbaColor=[0.5, 0.5, 0.5, 1])
        rock_body = p.createMultiBody(baseMass=0, baseCollisionShapeIndex=rock_shape, baseVisualShapeIndex=rock_visual, basePosition=[x, y, z],
baseOrientation=orientation)

def make_rocks(num_rocks=100, max_size=0.25, arena_size=10):
    for _ in range(num_rocks):
        x = random.uniform(-1 * arena_size/2, arena_size/2)
        y = random.uniform(-1 * arena_size/2, arena_size/2)
        z = 0.5 # Adjust based on your needs
        size = random.uniform(0.1, max_size)
        orientation = p.getQuaternionFromEuler([random.uniform(0, 3.14), random.uniform(0, 3.14), random.uniform(0, 3.14)])
```

```

    rock_shape = p.createCollisionShape(p.GEOM_BOX, halfExtents=[size, size, size])
    rock_visual = p.createVisualShape(p.GEOM_BOX, halfExtents=[size, size, size], rgbaColor=[0.5, 0.5, 0.5, 1])
    rock_body = p.createMultiBody(baseMass=0, baseCollisionShapeIndex=rock_shape, baseVisualShapeIndex=rock_visual, basePosition=[x, y, z],
baseOrientation=orientation)

def make_arena(arena_size=10, wall_height=1):
    wall_thickness = 0.5
    floor_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2, arena_size/2, wall_thickness])
    floor_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2, arena_size/2, wall_thickness], rgbaColor=[1, 1, 0, 1])
    floor_body = p.createMultiBody(baseMass=0, baseCollisionShapeIndex=floor_collision_shape, baseVisualShapeIndex=floor_visual_shape, basePosition=[0, 0, -
wall_thickness])

    wall_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2, wall_thickness/2, wall_height/2])
    wall_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX, halfExtents=[arena_size/2, wall_thickness/2, wall_height/2], rgbaColor=[0.7, 0.7, 0.7, 1]) # Gray walls

    # Create four walls
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape, baseVisualShapeIndex=wall_visual_shape, basePosition=[0, arena_size/2, wall_height/2])
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape, baseVisualShapeIndex=wall_visual_shape, basePosition=[0, -arena_size/2, wall_height/2])

    wall_collision_shape = p.createCollisionShape(shapeType=p.GEOM_BOX, halfExtents=[wall_thickness/2, arena_size/2, wall_height/2])
    wall_visual_shape = p.createVisualShape(shapeType=p.GEOM_BOX, halfExtents=[wall_thickness/2, arena_size/2, wall_height/2], rgbaColor=[0.7, 0.7, 0.7, 1]) # Gray walls

    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape, baseVisualShapeIndex=wall_visual_shape, basePosition=[arena_size/2, 0, wall_height/2])
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_collision_shape, baseVisualShapeIndex=wall_visual_shape, basePosition=[-arena_size/2, 0, wall_height/2])

p.setGravity(0, 0, -10)

arena_size = 20
make_arena(arena_size=arena_size)

#make_rocks(arena_size=arena_size)

mountain_position = (0, 0, -1) # Adjust as needed
mountain_orientation = p.getQuaternionFromEuler((0, 0, 0))
p.setAdditionalSearchPath('shapes/')
# mountain = p.loadURDF("mountain.urdf", mountain_position, mountain_orientation, useFixedBase=1)
# mountain = p.loadURDF("mountain_with_cubes.urdf", mountain_position, mountain_orientation, useFixedBase=1)

mountain = p.loadURDF("gaussian_pyramid.urdf", mountain_position, mountain_orientation, useFixedBase=1)

# generate a random creature
cr = creature.Creature(gene_count=3)
# save it to XML
with open('test.urdf', 'w') as f:
    f.write(cr.to_xml())
# load it into the sim
rob1 = p.loadURDF('test.urdf', (0, 0, 10))

p.setRealTimeSimulation(1)

```

--

src/demo_gui_elite.py — New (not in starter)

Yellow shading indicates differences relative to the starter code.

```
"""
demo_gui_elite.py

GUI demo/replay tool for Part B video evidence.

This script loads a saved elite DNA CSV (produced by run_ga_baseline.py) and replays
the creature in the mountain environment using PyBullet GUI.

Usage example:
python demo_gui_elite.py --elite_csv elite_dna/elite_gen_0049.csv --gene_count 3 --sim_steps 4800
"""

import argparse
import time

import genome
import creature
import simulation_mountain as simulation

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--elite_csv", type=str, required=True, help="Path to elite_gen_XXXX.csv DNA file")
    parser.add_argument("--gene_count", type=int, default=3, help="Gene count used to construct the Creature object")
    parser.add_argument("--sim_steps", type=int, default=2400, help="Simulation steps for the replay")
    parser.add_argument("--arena_size", type=float, default=20.0, help="Sandbox size")
    parser.add_argument("--wall_height", type=float, default=1.0, help="Arena wall height")
    parser.add_argument("--motor_force", type=float, default=20.0, help="Motor force during replay (video-friendly)")
    parser.add_argument("--loop", action="store_true", help="Replay continuously (useful for recording)")
    args = parser.parse_args()

    # Load DNA and build a creature
    dna = genome.Genome.from_csv(args.elite_csv)
    cr = creature.Creature(args.gene_count)
    cr.update_dna(dna)

    # GUI simulation for video
    sim = simulation.Simulation(arena_size=args.arena_size, wall_height=args.wall_height, gui=True)

    while True:
        # Replay in realtime and show metrics overlay for narration
        sim.run_creature(
            cr,
            iterations=args.sim_steps,
            motor_force=args.motor_force,
            real_time=True,
            show_debug=True,
```

```
)
```

```
if not args.loop:  
    break
```

```
# Small pause between loops so you can cut cleanly in the video  
time.sleep(0.5)
```

```
# Keep the window open briefly at the end (optional convenience)  
time.sleep(2.0)
```

```
if __name__ == "__main__":  
    main()
```

src/genome.py — Modified

Yellow shading indicates differences relative to the starter code.

```
import numpy as np
import copy
import random

class Genome():
    @staticmethod
    def get_random_gene(length):
        gene = np.array([np.random.random() for i in range(length)])
        return gene

    @staticmethod
    def get_random_genome(gene_length, gene_count):
        genome = [Genome.get_random_gene(gene_length) for i in range(gene_count)]
        return genome

    @staticmethod
    def get_gene_spec():
        gene_spec = {"link-shape": {"scale": 1},
                    "link-length": {"scale": 2},
                    "link-radius": {"scale": 1},
                    "link-recurrence": {"scale": 3},
                    "link-mass": {"scale": 1},
                    "joint-type": {"scale": 1},
                    "joint-parent": {"scale": 1},
                    "joint-axis-xyz": {"scale": 1},
                    "joint-origin-rpy-1": {"scale": np.pi * 2},
                    "joint-origin-rpy-2": {"scale": np.pi * 2},
                    "joint-origin-rpy-3": {"scale": np.pi * 2},
                    "joint-origin-xyz-1": {"scale": 1},
                    "joint-origin-xyz-2": {"scale": 1},
                    "joint-origin-xyz-3": {"scale": 1},
                    "control-waveform": {"scale": 1},
                    "control-amp": {"scale": 0.25},
                    "control-freq": {"scale": 1}
                    }
        ind = 0
        for key in gene_spec.keys():
            gene_spec[key]["ind"] = ind
            ind = ind + 1
        return gene_spec

    @staticmethod
    def get_gene_dict(gene, spec):
        gdict = {}
        for key in spec:
            ind = spec[key]["ind"]
            scale = spec[key]["scale"]
```

```

    gdict[key] = gene[ind] * scale
    return gdict

@staticmethod
def get_genome_dicts(genome, spec):
    gdicts = []
    for gene in genome:
        gdicts.append(Genome.get_gene_dict(gene, spec))
    return gdicts

@staticmethod
def expandLinks(parent_link, uniq_parent_name, flat_links, exp_links):
    children = [l for l in flat_links if l.parent_name == parent_link.name]
    sibling_ind = 1
    for c in children:
        for r in range(int(c.recur)):
            sibling_ind = sibling_ind + 1
            c_copy = copy.copy(c)
            c_copy.parent_name = uniq_parent_name
            uniq_name = c_copy.name + str(len(exp_links))
            #print("exp: ", c.name, " -> ", uniq_name)
            c_copy.name = uniq_name
            c_copy.sibling_ind = sibling_ind
            exp_links.append(c_copy)
            assert c.parent_name != c.name, "Genome::expandLinks: link joined to itself: " + c.name + " joins " + c.parent_name
    Genome.expandLinks(c, uniq_name, flat_links, exp_links)

@staticmethod
def genome_to_links(gdicts):
    links = []
    link_ind = 0
    parent_names = [str(link_ind)]
    for gdict in gdicts:
        link_name = str(link_ind)
        parent_ind = gdict["joint-parent"] * len(parent_names)
        assert parent_ind < len(parent_names), "genome.py: parent ind too high: " + str(parent_ind) + "got: " + str(parent_names)
        parent_name = parent_names[int(parent_ind)]
        #print("available parents: ", parent_names, "chose", parent_name)
        recur = gdict["link-recurrence"]
        link = URDFLink(name=link_name,
                        parent_name=parent_name,
                        recur=recur+1,
                        link_length=gdict["link-length"],
                        link_radius=gdict["link-radius"],
                        link_mass=gdict["link-mass"],
                        joint_type=gdict["joint-type"],
                        joint_parent=gdict["joint-parent"],
                        joint_axis_xyz=gdict["joint-axis-xyz"],
                        joint_origin_rpy_1=gdict["joint-origin-rpy-1"],
                        joint_origin_rpy_2=gdict["joint-origin-rpy-2"],
                        joint_origin_rpy_3=gdict["joint-origin-rpy-3"],
                        joint_origin_xyz_1=gdict["joint-origin-xyz-1"],

```

```

        joint_origin_xyz_2=gdict["joint-origin-xyz-2"],
        joint_origin_xyz_3=gdict["joint-origin-xyz-3"],
        control_waveform=gdict["control-waveform"],
        control_amp=gdict["control-amp"],
        control_freq=gdict["control-freq"])
links.append(link)
if link_ind != 0:# don't re-add the first link
    parent_names.append(link_name)
link_ind = link_ind + 1

# now just fix the first link so it links to nothing
links[0].parent_name = "None"
return links

@staticmethod
def crossover(g1, g2):
    x1 = random.randint(0, len(g1)-1)
    x2 = random.randint(0, len(g2)-1)
    g3 = np.concatenate((g1[x1:], g2[x2:]))
    if len(g3) > len(g1):
        g3 = g3[0:len(g1)]
    return g3

@staticmethod
def point_mutate(genome, rate, amount):
    """
    Point mutation:
    - With probability `rate` per gene value, add a random delta in [-amount, +amount]
    - Clamp values into [0.0, 0.9999]
    - Deep-copy genes so the original genome is not modified
    """
    amt = float(amount)
    if amt < 0:
        amt = -amt

    # Copy inner lists too (important: avoid mutating the parent genome)
    new_genome = [gene.copy() for gene in genome]

    for gene in new_genome:
        for i, val in enumerate(gene):
            if random.random() < rate:
                delta = random.uniform(-amt, amt)
                val = val + delta

        # Clamp to valid range
        if val >= 1.0:
            val = 0.9999
        elif val < 0.0:
            val = 0.0

        gene[i] = val

```

```

return new_genome

@staticmethod
def shrink_mutate(genome, rate):
    if len(genome) == 1:
        return copy.copy(genome)
    if random.random() < rate:
        ind = random.randint(0, len(genome)-1)
        new_genome = np.delete(genome, ind, 0)
        return new_genome
    else:
        return copy.copy(genome)

@staticmethod
def grow_mutate(genome, rate):
    if random.random() < rate:
        gene = Genome.get_random_gene(len(genome[0]))
        new_genome = copy.copy(genome)
        new_genome = np.append(new_genome, [gene], axis=0)
        return new_genome
    else:
        return copy.copy(genome)

@staticmethod
def to_csv(dna, csv_file):
    csv_str = ""
    for gene in dna:
        for val in gene:
            csv_str = csv_str + str(val) + ","
        csv_str = csv_str + '\n'

    with open(csv_file, 'w') as f:
        f.write(csv_str)

@staticmethod
def from_csv(filename):
    csv_str = ""
    with open(filename) as f:
        csv_str = f.read()
    dna = []
    lines = csv_str.split('\n')
    for line in lines:
        vals = line.split(',')
        gene = [float(v) for v in vals if v != ""]
        if len(gene) > 0:
            dna.append(np.array(gene, dtype=float))
    return dna

class URDFLink:
    def __init__(self, name, parent_name, recur,
                 link_length=0.1,

```

```

        link_radius=0.1,
        link_mass=0.1,
        joint_type=0.1,
        joint_parent=0.1,
        joint_axis_xyz=0.1,
        joint_origin_rpy_1=0.1,
        joint_origin_rpy_2=0.1,
        joint_origin_rpy_3=0.1,
        joint_origin_xyz_1=0.1,
        joint_origin_xyz_2=0.1,
        joint_origin_xyz_3=0.1,
        control_waveform=0.1,
        control_amp=0.1,
        control_freq=0.1):
    self.name = name
    self.parent_name = parent_name
    self.recur = recur
    self.link_length=link_length
    self.link_radius=link_radius
    self.link_mass=link_mass
    self.joint_type=joint_type
    self.joint_parent=joint_parent
    self.joint_axis_xyz=joint_axis_xyz
    self.joint_origin_rpy_1=joint_origin_rpy_1
    self.joint_origin_rpy_2=joint_origin_rpy_2
    self.joint_origin_rpy_3=joint_origin_rpy_3
    self.joint_origin_xyz_1=joint_origin_xyz_1
    self.joint_origin_xyz_2=joint_origin_xyz_2
    self.joint_origin_xyz_3=joint_origin_xyz_3
    self.control_waveform=control_waveform
    self.control_amp=control_amp
    self.control_freq=control_freq
    self.sibling_ind = 1

def to_link_element(self, adom):
    # <link name="base_link">
    # <visual>
    # <geometry>
    # <cylinder length="0.6" radius="0.25"/>
    # </geometry>
    # </visual>
    # <collision>
    # <geometry>
    # <cylinder length="0.6" radius="0.25"/>
    # </geometry>
    # </collision>
    # <inertial>
    # <mass value="0.25"/>
    # <inertia ixx="0.0003" iyy="0.0003" izz="0.0003" ixy="0" ixz="0" iyz="0"/>
    # </inertial>
    # </link>

```

```

link_tag = adom.createElement("link")
link_tag.setAttribute("name", self.name)
vis_tag = adom.createElement("visual")
geom_tag = adom.createElement("geometry")
cyl_tag = adom.createElement("cylinder")
cyl_tag.setAttribute("length", str(self.link_length))
cyl_tag.setAttribute("radius", str(self.link_radius))

geom_tag.appendChild(cyl_tag)
vis_tag.appendChild(geom_tag)

coll_tag = adom.createElement("collision")
c_geom_tag = adom.createElement("geometry")
c_cyl_tag = adom.createElement("cylinder")
c_cyl_tag.setAttribute("length", str(self.link_length))
c_cyl_tag.setAttribute("radius", str(self.link_radius))

c_geom_tag.appendChild(c_cyl_tag)
coll_tag.appendChild(c_geom_tag)

# <inertial>
# <mass value="0.25"/>
# <inertia ixx="0.0003" iyy="0.0003" izz="0.0003" ixy="0" ixz="0" iyz="0"/>
# </inertial>
inertial_tag = adom.createElement("inertial")
mass_tag = adom.createElement("mass")
# pi r^2 * height
mass = np.pi * (self.link_radius * self.link_radius) * self.link_length
mass_tag.setAttribute("value", str(mass))
inertia_tag = adom.createElement("inertia")
# <inertia ixx="0.0003" iyy="0.0003" izz="0.0003" ixy="0" ixz="0" iyz="0"/>
inertia_tag.setAttribute("ixx", "0.03")
inertia_tag.setAttribute("iyy", "0.03")
inertia_tag.setAttribute("izz", "0.03")
inertia_tag.setAttribute("ixy", "0")
inertia_tag.setAttribute("ixz", "0")
inertia_tag.setAttribute("iyx", "0")
inertial_tag.appendChild(mass_tag)
inertial_tag.appendChild(inertia_tag)

link_tag.appendChild(vis_tag)
link_tag.appendChild(coll_tag)
link_tag.appendChild(inertial_tag)

return link_tag

def to_joint_element(self, adom):
# <joint name="base_to_sub2" type="revolute">
# <parent link="base_link"/>
# <child link="sub_link2"/>

```

```

# <axis xyz="1 0 0"/>
# <limit effort="10" upper="0" lower="10" velocity="1"/>
# <origin rpy="0 0 0" xyz="0 0.5 0"/>
# </joint>
joint_tag = adom.createElement("joint")
joint_tag.setAttribute("name", self.name + "_to_" + self.parent_name)
if self.joint_type >= 0.5:
    joint_tag.setAttribute("type", "revolute")
else:
    joint_tag.setAttribute("type", "revolute")
parent_tag = adom.createElement("parent")
parent_tag.setAttribute("link", self.parent_name)
child_tag = adom.createElement("child")
child_tag.setAttribute("link", self.name)
axis_tag = adom.createElement("axis")
if self.joint_axis_xyz <= 0.33:
    axis_tag.setAttribute("xyz", "1 0 0")
if self.joint_axis_xyz > 0.33 and self.joint_axis_xyz <= 0.66:
    axis_tag.setAttribute("xyz", "0 1 0")
if self.joint_axis_xyz > 0.66:
    axis_tag.setAttribute("xyz", "0 0 1")

limit_tag = adom.createElement("limit")
# effort upper lower velocity
limit_tag.setAttribute("effort", "1")

# PyBullet expects lower < upper; reversed limits can lock joints and prevent movement.
limit_tag.setAttribute("upper", "3.1415")
limit_tag.setAttribute("lower", "-3.1415")

limit_tag.setAttribute("velocity", "1")

# <origin rpy="0 0 0" xyz="0 0.5 0"/>
orig_tag = adom.createElement("origin")

rpy1 = self.joint_origin_rpy_1 * self.sibling_ind
rpy = str(rpy1) + " " + str(self.joint_origin_rpy_2) + " " + str(self.joint_origin_rpy_3)

orig_tag.setAttribute("rpy", rpy)
xyz = str(self.joint_origin_xyz_1) + " " + str(self.joint_origin_xyz_2) + " " + str(self.joint_origin_xyz_3)
orig_tag.setAttribute("xyz", xyz)

joint_tag.appendChild(parent_tag)
joint_tag.appendChild(child_tag)
joint_tag.appendChild(axis_tag)
joint_tag.appendChild(limit_tag)
joint_tag.appendChild(orig_tag)
return joint_tag

```



src/motor_test.py — Unchanged (starter)

Yellow shading indicates differences relative to the starter code.

```
import genome
import creature
import pybullet as p
import time
import random
import numpy as np

## ... usual starter code to create a sim and floor
p.connect(p.GUI)
p.setPhysicsEngineParameter(enableFileCaching=0)
p.configureDebugVisualizer(p.COV_ENABLE_GUI, 0)
plane_shape = p.createCollisionShape(p.GEOM_PLANE)
floor = p.createMultiBody(plane_shape, plane_shape)
p.setGravity(0, 0, -10)
# commenting out for mac compatibility
#p.setRealTimeSimulation(1)

# generate a random creature
cr = creature.Creature(gene_count=3)
# save it to XML
with open('test.urdf', 'w') as f:
    f.write(cr.to_xml())
# load it into the sim
rob1 = p.loadURDF('test.urdf')
start_pos, orn = p.getBasePositionAndOrientation(rob1)

# iterate
elapsed_time = 0
wait_time = 1.0 / 240 # seconds
total_time = 5 # seconds
step = 0
dist_moved = 0
while True:
    p.stepSimulation()
    step += 1
    if step % 120 == 0:
        motors = cr.get_motors()
        assert len(motors) == p.getNumJoints(rob1), "Something went wrong"
        for jid in range(p.getNumJoints(rob1)):
            mode = p.VELOCITY_CONTROL
            vel = motors[jid].get_output()
            p.setJointMotorControl2(rob1,
                                   jid,
                                   controlMode=mode,
                                   targetVelocity=vel)
        new_pos, orn = p.getBasePositionAndOrientation(rob1)
        #print(new_pos)
```

```
    dist_moved = np.linalg.norm(np.asarray(start_pos) - np.asarray(new_pos))
    print(dist_moved)
    time.sleep(wait_time)
    elapsed_time += wait_time
    if elapsed_time > total_time:
        break

print("TOTAL DISTANCE MOVED:", dist_moved)
```

src/offline_from_csv.py — Unchanged (starter)

Yellow shading indicates differences relative to the starter code.

```
import os
import genome
import sys
import creature
import pybullet as p
import time
import random
import numpy as np

## ... usual starter code to create a sim and floor
def main(csv_file):
    assert os.path.exists(csv_file), "Tried to load " + csv_file + " but it does not exists"

    p.connect(p.DIRECT)
    p.setPhysicsEngineParameter(enableFileCaching=0)
    p.configureDebugVisualizer(p.COV_ENABLE_GUI, 0)
    plane_shape = p.createCollisionShape(p.GEOM_PLANE)
    floor = p.createMultiBody(plane_shape, plane_shape)
    p.setGravity(0, 0, -10)
    # p.setRealTimeSimulation(1)

    # generate a random creature
    cr = creature.Creature(gene_count=1)
    dna = genome.Genome.from_csv(csv_file)
    cr.update_dna(dna)
    # save it to XML
    with open('test.urdf', 'w') as f:
        f.write(cr.to_xml())
    # load it into the sim
    rob1 = p.loadURDF('test.urdf')
    # air drop it
    p.resetBasePositionAndOrientation(rob1, [0, 0, 2.5], [0, 0, 0, 1])

    start_pos, orn = p.getBasePositionAndOrientation(rob1)

    # iterate
    elapsed_time = 0
    wait_time = 1.0/240 # seconds
    total_time = 30 # seconds
    step = 0
    while True:
        p.stepSimulation()
        step += 1
        if step % 24 == 0:
            motors = cr.get_motors()
            assert len(motors) == p.getNumJoints(rob1), "Something went wrong"
```

```
for jid in range(p.getNumJoints(rob1)):
    mode = p.VELOCITY_CONTROL
    vel = motors[jid].get_output()
    p.setJointMotorControl2(rob1,
        jid,
        controlMode=mode,
        targetVelocity=vel)
    new_pos, orn = p.getBasePositionAndOrientation(rob1)
    #print(new_pos)
    dist_moved = np.linalg.norm(np.asarray(start_pos) - np.asarray(new_pos))
    print(dist_moved)
    #time.sleep(wait_time)
    elapsed_time += wait_time
    if elapsed_time > total_time:
        break

print("TOTAL DISTANCE MOVED:", dist_moved)

if __name__ == "__main__":
    assert len(sys.argv) == 2, "Usage: python playback_test.py csv_filename"
    main(sys.argv[1])
```

src/population.py — Unchanged (starter)

Yellow shading indicates differences relative to the starter code.

```
import creature
import numpy as np

class Population:
    def __init__(self, pop_size, gene_count):
        self.creatures = [creature.Creature(
            gene_count=gene_count)
            for i in range(pop_size)]

    @staticmethod
    def get_fitness_map(fits):
        fitmap = []
        total = 0
        for f in fits:
            total = total + f
            fitmap.append(total)
        return fitmap

    @staticmethod
    def select_parent(fitmap):
        r = np.random.rand() # 0-1
        r = r * fitmap[-1]
        for i in range(len(fitmap)):
            if r <= fitmap[i]:
                return i
```

src/prepare_shapes.py — Unchanged (starter)

Yellow shading indicates differences relative to the starter code.

```
import random
import math
from noise import pnoise2

# def write_to_obj(filename, vertices, indices):
#     with open(filename, 'w') as f:
#         # Write vertices to the file
#         for vertex in vertices:
#             f.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
#
#         # Write faces to the file
#         for face in indices:
#             # OBJ indices start from 1, not 0
#             f.write(f"f {face[0]+1} {face[1]+1} {face[2]+1}\n")

def write_to_obj(filename, vertices, indices):
    with open(filename, 'w') as f:
        # Write vertices to the file
        for vertex in vertices:
            f.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")

        # Write faces to the file
        for face in indices:
            f.write(f"f {face[0]+1} {face[1]+1} {face[2]+1}\n")

def make_pyramid(filename):
    # Define the tetrahedron vertices and indices
    vertices = [
        [-5, -5, 0], # Base vertex 1
        [5, -5, 0],  # Base vertex 2
        [0, 5, 0],   # Base vertex 3
        [0, 0, 5]    # Apex vertex
    ]
    indices = [
        [0, 1, 2], # Base triangle
        [0, 1, 3], # Side triangle 1
        [1, 2, 3], # Side triangle 2
        [2, 0, 3]  # Side triangle 3
    ]
    # Write the tetrahedron to an OBJ file
    write_to_obj(filename, vertices, indices)

def make_rocky_mountain(filename):

    # Define the pyramid vertices and indices
    vertices = [
```

```

[-5, -5, 0], # Base vertex 1
[5, -5, 0], # Base vertex 2
[5, 5, 0], # Base vertex 3
[-5, 5, 0], # Base vertex 4
[0, 0, 5] # Apex vertex
]
indices = [
    [0, 1, 4], # Side triangle 1
    [1, 2, 4], # Side triangle 2
    [2, 3, 4], # Side triangle 3
    [3, 0, 4], # Side triangle 4
    [0, 1, 2], # Base triangle 1
    [2, 3, 0] # Base triangle 2
]

# Add random cubes embedded in the pyramid
num_cubes = 5
for _ in range(num_cubes):
    size = random.uniform(0.5, 2)
    x = random.uniform(-4, 4)
    y = random.uniform(-4, 4)
    z = random.uniform(0.5, 4.5)

    # Define cube vertices
    cube_vertices = [
        [x-size/2, y-size/2, z-size/2],
        [x+size/2, y-size/2, z-size/2],
        [x+size/2, y+size/2, z-size/2],
        [x-size/2, y+size/2, z-size/2],
        [x-size/2, y-size/2, z+size/2],
        [x+size/2, y-size/2, z+size/2],
        [x+size/2, y+size/2, z+size/2],
        [x-size/2, y+size/2, z+size/2]
    ]

    # Define cube faces
    cube_indices = [
        [0, 1, 2], [2, 3, 0], # Bottom
        [4, 5, 6], [6, 7, 4], # Top
        [0, 1, 5], [5, 4, 0], # Front
        [1, 2, 6], [6, 5, 1], # Right
        [2, 3, 7], [7, 6, 2], # Back
        [3, 0, 4], [4, 7, 3] # Left
    ]

    # Offset cube indices by the current number of vertices
    offset = len(vertices)
    cube_indices = [[i+offset for i in face] for face in cube_indices]

    # Add cube vertices and indices to the main lists
    vertices.extend(cube_vertices)
    indices.extend(cube_indices)

```

```

# Write the combined vertices and indices to an OBJ file
write_to_obj(filename, vertices, indices)

def gaussian(x, y, sigma):
    """Return the height of the shape at position (x, y) using a Gaussian function."""
    return math.exp(-((x**2 + y**2) / (2 * sigma**2)))

def generate_gaussian_pyramid(filename, size=10, resolution=0.5, sigma=3):
    vertices = []
    faces = []

    # Generate vertices using the Gaussian function
    for i in range(int(size / resolution)):
        for j in range(int(size / resolution)):
            x = -size/2 + i * resolution
            y = -size/2 + j * resolution
            z = gaussian(x, y, sigma)
            vertices.append([x, y, z])

    # Generate faces by connecting vertices
    for i in range(int(size / resolution) - 1):
        for j in range(int(size / resolution) - 1):
            # Calculate indices for the current square
            bottom_left = i * int(size / resolution) + j
            bottom_right = i * int(size / resolution) + j + 1
            top_left = (i + 1) * int(size / resolution) + j
            top_right = (i + 1) * int(size / resolution) + j + 1

            # Create two triangles for the current square
            faces.append([bottom_left, bottom_right, top_left])
            faces.append([top_left, bottom_right, top_right])

    # Write to OBJ file
    with open(filename, 'w') as f:
        for vertex in vertices:
            f.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
        for face in faces:
            f.write(f"f {face[0]+1} {face[1]+1} {face[2]+1}\n")

import math

def gaussian2(x, y, sigma, height):
    """Return the height of the shape at position (x, y) using a Gaussian function."""
    return height * math.exp(-((x**2 + y**2) / (2 * sigma**2)))

def generate_gaussian_pyramid2(filename, size=10, resolution=0.5, sigma=3, height=5):
    vertices = []
    faces = []

```

```

# Generate vertices using the Gaussian function for the pyramid
for i in range(int(size / resolution)):
    for j in range(int(size / resolution)):
        x = -size/2 + i * resolution
        y = -size/2 + j * resolution
        z = gaussian2(x, y, sigma, height)
        vertices.append([x, y, z])

# Generate vertices for the flat base
for i in range(int(size / resolution)):
    for j in range(int(size / resolution)):
        x = -size/2 + i * resolution
        y = -size/2 + j * resolution
        z = 0
        vertices.append([x, y, z])

# Generate faces by connecting vertices
for i in range(int(size / resolution) - 1):
    for j in range(int(size / resolution) - 1):
        # Calculate indices for the current square on the pyramid
        bottom_left = i * int(size / resolution) + j
        bottom_right = i * int(size / resolution) + j + 1
        top_left = (i + 1) * int(size / resolution) + j
        top_right = (i + 1) * int(size / resolution) + j + 1

        # Create two triangles for the current square on the pyramid
        faces.append([bottom_left, bottom_right, top_left])
        faces.append([top_left, bottom_right, top_right])

        # Calculate indices for the current square on the base
        base_offset = int(size / resolution) * int(size / resolution)
        bottom_left_base = base_offset + bottom_left
        bottom_right_base = base_offset + bottom_right
        top_left_base = base_offset + top_left
        top_right_base = base_offset + top_right

        # Create two triangles for the current square on the base
        faces.append([bottom_left_base, top_left_base, bottom_right_base])
        faces.append([top_left_base, top_right_base, bottom_right_base])

# Write to OBJ file
with open(filename, 'w') as f:
    for vertex in vertices:
        f.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
    for face in faces:
        f.write(f"f {face[0]+1} {face[1]+1} {face[2]+1}\n")

def generate_gaussian_pyramid3(filename, size=10, resolution=0.5, sigma=3, height=5, noise_scale=0.5, noise_factor=1.0):
    """
    pyramid with noise

```

```
"""
```

```
vertices = []
```

```
faces = []
```

```
# Generate vertices using the Gaussian function for the pyramid
```

```
for i in range(int(size / resolution)):
```

```
    for j in range(int(size / resolution)):
```

```
        x = -size/2 + i * resolution
```

```
        y = -size/2 + j * resolution
```

```
        z = gaussian2(x, y, sigma, height)
```

```
        # Add Perlin noise to the z-coordinate
```

```
        z += pnoise2(x * noise_scale, y * noise_scale) * noise_factor
```

```
        vertices.append([x, y, z])
```

```
# Generate vertices for the flat base
```

```
for i in range(int(size / resolution)):
```

```
    for j in range(int(size / resolution)):
```

```
        x = -size/2 + i * resolution
```

```
        y = -size/2 + j * resolution
```

```
        z = 0
```

```
        vertices.append([x, y, z])
```

```
# Generate faces by connecting vertices
```

```
for i in range(int(size / resolution) - 1):
```

```
    for j in range(int(size / resolution) - 1):
```

```
        # Calculate indices for the current square on the pyramid
```

```
        bottom_left = i * int(size / resolution) + j
```

```
        bottom_right = i * int(size / resolution) + j + 1
```

```
        top_left = (i + 1) * int(size / resolution) + j
```

```
        top_right = (i + 1) * int(size / resolution) + j + 1
```

```
        # Create two triangles for the current square on the pyramid
```

```
        faces.append([bottom_left, bottom_right, top_left])
```

```
        faces.append([top_left, bottom_right, top_right])
```

```
        # Calculate indices for the current square on the base
```

```
        base_offset = int(size / resolution) * int(size / resolution)
```

```
        bottom_left_base = base_offset + bottom_left
```

```
        bottom_right_base = base_offset + bottom_right
```

```
        top_left_base = base_offset + top_left
```

```
        top_right_base = base_offset + top_right
```

```
        # Create two triangles for the current square on the base
```

```
        faces.append([bottom_left_base, top_left_base, bottom_right_base])
```

```
        faces.append([top_left_base, top_right_base, bottom_right_base])
```

```
# Write to OBJ file
```

```
with open(filename, 'w') as f:
```

```
    for vertex in vertices:
```

```
        f.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
```

```
for face in faces:
    f.write(f"{face[0]+1} {face[1]+1} {face[2]+1}\n")
```

```
def generate_gaussian_pyramid4(filename, size=10, resolution=0.5, sigma=3, height=5, noise_scale=0.5, noise_factor=1.0):
```

```
    vertices = []
    faces = []
```

```
    # Generate vertices using the Gaussian function for the top surface of the pyramid
```

```
    for i in range(int(size / resolution)):
```

```
        for j in range(int(size / resolution)):
```

```
            x = -size/2 + i * resolution
```

```
            y = -size/2 + j * resolution
```

```
            z = gaussian2(x, y, sigma, height)
```

```
            # Add Perlin noise to the z-coordinate
```

```
            z += pnoise2(x * noise_scale, y * noise_scale) * noise_factor
```

```
            vertices.append([x, y, z])
```

```
    # Generate vertices for the bottom surface (flat base) of the pyramid
```

```
    for i in range(int(size / resolution)):
```

```
        for j in range(int(size / resolution)):
```

```
            x = -size/2 + i * resolution
```

```
            y = -size/2 + j * resolution
```

```
            z = 0
```

```
            vertices.append([x, y, z])
```

```
    # Generate faces for the top surface
```

```
    for i in range(int(size / resolution) - 1):
```

```
        for j in range(int(size / resolution) - 1):
```

```
            bottom_left = i * int(size / resolution) + j
```

```
            bottom_right = i * int(size / resolution) + j + 1
```

```
            top_left = (i + 1) * int(size / resolution) + j
```

```
            top_right = (i + 1) * int(size / resolution) + j + 1
```

```
            faces.append([bottom_left, bottom_right, top_left])
```

```
            faces.append([top_left, bottom_right, top_right])
```

```
    # Generate faces for the bottom surface
```

```
    base_offset = int(size / resolution) * int(size / resolution)
```

```
    for i in range(int(size / resolution) - 1):
```

```
        for j in range(int(size / resolution) - 1):
```

```
            bottom_left_base = base_offset + i * int(size / resolution) + j
```

```
            bottom_right_base = base_offset + i * int(size / resolution) + j + 1
```

```
            top_left_base = base_offset + (i + 1) * int(size / resolution) + j
```

```
            top_right_base = base_offset + (i + 1) * int(size / resolution) + j + 1
```

```
            faces.append([bottom_left_base, top_left_base, bottom_right_base])
```

```
            faces.append([top_left_base, top_right_base, bottom_right_base])
```

```

# Generate faces for the sides
for i in range(int(size / resolution) - 1):
    for j in range(int(size / resolution)):
        top = i * int(size / resolution) + j
        bottom = base_offset + i * int(size / resolution) + j
        top_next = (i + 1) * int(size / resolution) + j
        bottom_next = base_offset + (i + 1) * int(size / resolution) + j

        faces.append([top, bottom, top_next])
        faces.append([top_next, bottom, bottom_next])

# Write to OBJ file
with open(filename, 'w') as f:
    for vertex in vertices:
        f.write(f"v {vertex[0]} {vertex[1]} {vertex[2]}\n")
    for face in faces:
        f.write(f"f {face[0]+1} {face[1]+1} {face[2]+1}\n")

# Generate the OBJ file
generate_gaussian_pyramid4("./shapes/gaussian_pyramid.obj")

# Generate the OBJ file
# generate_gaussian_pyramid3("./shapes/gaussian_pyramid.obj")

# Generate the OBJ file
# generate_gaussian_pyramid("./shapes/gaussian_pyramid.obj")

# make_pyramid('./shapes/pyramid.obj')
# make_rocky_mountain('./shapes/mountain_with_cubes.obj')

# make_pyramid('mountain.obj')
# make_rocky_mountain('./shapes/mountain_with_cubes.obj')

```

src/realtime_from_csv.py — Modified

Yellow shading indicates differences relative to the starter code.

```
import os
import sys
import time
import numpy as np
import pybullet as p

import genome
import creature

def make_arena(pid, arena_size=20.0, wall_height=1.0):
    wall_thickness = 0.5

    # Floor (big box)
    floor_shape = p.createCollisionShape(
        shapeType=p.GEOM_BOX,
        halfExtents=[arena_size / 2, arena_size / 2, wall_thickness],
        physicsClientId=pid
    )
    floor_id = p.createMultiBody(
        baseMass=0,
        baseCollisionShapeIndex=floor_shape,
        basePosition=[0, 0, -wall_thickness],
        physicsClientId=pid
    )

    # Two walls along Y (north/south)
    wall_shape_y = p.createCollisionShape(
        shapeType=p.GEOM_BOX,
        halfExtents=[arena_size / 2, wall_thickness / 2, wall_height / 2],
        physicsClientId=pid
    )
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_shape_y,
        basePosition=[0, arena_size / 2, wall_height / 2], physicsClientId=pid)
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_shape_y,
        basePosition=[0, -arena_size / 2, wall_height / 2], physicsClientId=pid)

    # Two walls along X (east/west)
    wall_shape_x = p.createCollisionShape(
        shapeType=p.GEOM_BOX,
        halfExtents=[wall_thickness / 2, arena_size / 2, wall_height / 2],
        physicsClientId=pid
    )
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_shape_x,
        basePosition=[arena_size / 2, 0, wall_height / 2], physicsClientId=pid)
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_shape_x,
        basePosition=[-arena_size / 2, 0, wall_height / 2], physicsClientId=pid)
```

```
return floor_id
```

```
def load_mountain(pid):
```

```
    shapes_path = os.path.join(os.path.dirname(__file__), "shapes")
```

```
    p.setAdditionalSearchPath(shapes_path, physicsClientId=pid)
```

```
    mountain_position = (0, 0, -1)
```

```
    mountain_orientation = p.getQuaternionFromEuler((0, 0, 0))
```

```
    mountain_id = p.loadURDF(
```

```
        "gaussian_pyramid.urdf",
```

```
        mountain_position,
```

```
        mountain_orientation,
```

```
        useFixedBase=1,
```

```
        physicsClientId=pid
```

```
    )
```

```
    return mountain_id
```

```
def main(csv_file):
```

```
    assert os.path.exists(csv_file), f"Tried to load {csv_file} but it does not exist"
```

```
    pid = p.connect(p.GUI)
```

```
    p.setPhysicsEngineParameter(enableFileCaching=0, physicsClientId=pid)
```

```
    p.configureDebugVisualizer(p.COV_ENABLE_GUI, 0, physicsClientId=pid)
```

```
    p.resetSimulation(physicsClientId=pid)
```

```
    p.setGravity(0, 0, -10, physicsClientId=pid)
```

```
    arena_size = 20.0
```

```
    make_arena(pid, arena_size=arena_size, wall_height=1.0)
```

```
    load_mountain(pid)
```

```
    # Camera (so you actually see the mountain)
```

```
    p.resetDebugVisualizerCamera(
```

```
        cameraDistance=14,
```

```
        cameraYaw=35,
```

```
        cameraPitch=-35,
```

```
        cameraTargetPosition=(0, 0, 2),
```

```
        physicsClientId=pid
```

```
    )
```

```
    # Load genome from CSV (this must be an elite_gen_XXXX.csv, not ga_baseline_log.csv)
```

```
    dna = genome.Genome.from_csv(csv_file)
```

```
    cr = creature.Creature(gene_count=1)
```

```
    cr.update_dna(dna)
```

```
    # Write creature URDF next to this script (stable path)
```

```
    urdf_path = os.path.join(os.path.dirname(__file__), "temp_replay.urdf")
```

```
    with open(urdf_path, "w", encoding="utf-8") as f:
```

```
        f.write(cr.to_xml())
```

```

rob = p.loadURDF(urdf_path, physicsClientId=pid)

# Spawn near the edge (same idea as simulation_mountain.py)
spawn_pos = [0, -arena_size / 2 + 2.0, 1.5]
p.resetBasePositionAndOrientation(rob, spawn_pos, [0, 0, 0, 1], physicsClientId=pid)
start_pos, _ = p.getBasePositionAndOrientation(rob, physicsClientId=pid)

wait_time = 1.0 / 240.0
total_time = 60.0 # seconds (increase as needed)
elapsed = 0.0
step = 0
dist_moved = 0.0

try:
    while p.isConnected(physicsClientId=pid) and elapsed < total_time:
        p.stepSimulation(physicsClientId=pid)
        step += 1

        if step % 24 == 0:
            motors = cr.get_motors()
            assert len(motors) == p.getNumJoints(rob, physicsClientId=pid), "Motor/joint mismatch"

            for jid in range(p.getNumJoints(rob, physicsClientId=pid)):
                vel = motors[jid].get_output()
                p.setJointMotorControl2(
                    rob,
                    jid,
                    controlMode=p.VELOCITY_CONTROL,
                    targetVelocity=vel,
                    force=5,
                    physicsClientId=pid
                )

            new_pos, _ = p.getBasePositionAndOrientation(rob, physicsClientId=pid)
            dist_moved = float(np.linalg.norm(np.asarray(start_pos) - np.asarray(new_pos)))
            print(dist_moved)

            time.sleep(wait_time)
            elapsed += wait_time

except KeyboardInterrupt:
    pass
finally:
    if p.isConnected(physicsClientId=pid):
        p.disconnect(physicsClientId=pid)

print("TOTAL DISTANCE MOVED:", dist_moved)

if __name__ == "__main__":
    assert len(sys.argv) == 2, "Usage: python realtime_from_csv.py elite_gen_XXXX.csv"
    main(sys.argv[1])

```



src/run_ga_baseline.py — New (not in starter)

Yellow shading indicates differences relative to the starter code.

```
import argparse
import csv
import os
import time
import numpy as np

import random

import population
import simulation_mountain as simulation
import genome
import creature

def _select_parent_index(fit_map, pop_size):
    """
    Roulette selection, but falls back to random selection if total fitness is 0.
    """
    if len(fit_map) == 0 or fit_map[-1] <= 0:
        return np.random.randint(0, pop_size)
    return population.Population.select_parent(fit_map)

def run_ga(
    pop_size=10,
    gene_count=3,
    generations=100,
    sim_steps=2400,
    point_rate=0.1,
    point_amount=0.25,
    shrink_rate=0.25,
    grow_rate=0.1,
    seed=1,
    out_csv="ga_baseline_log.csv",
    elite_dir="elite_dna",
):
    np.random.seed(seed)
    # Ensure reproducibility across both NumPy and Python's random module (Genome uses random).
    random.seed(seed)

    os.makedirs(elite_dir, exist_ok=True)

    pop = population.Population(pop_size=pop_size, gene_count=gene_count)
    sim = simulation.Simulation()

    # CSV logging (generation-level)
    with open(out_csv, "w", newline="") as f:
```

```

writer = csv.writer(f)
writer.writerow([
    "generation",
    "max_fitness",
    "mean_fitness",
    "max_links",
    "mean_links",

    # Mountain-specific metrics (Part B evidence)
    "best_peak_dist", # peak distance of best-by-fitness creature
    "mean_peak_dist",
    "best_max_height", # max height of best-by-fitness creature
    "mean_max_height",
    "best_air_ratio", # airborne fraction (lower is better)
    "mean_air_ratio",
    "best_supported_ratio", # supported fraction (higher is better)
    "mean_supported_ratio",

    "time_sec"
])

for gen in range(generations):
    t0 = time.time()

    # Evaluate all creatures on the mountain
    for cr in pop.creatures:
        sim.run_creature(cr, sim_steps)

    # Part B fitness: closeness-to-peak with anti-cheat (supported vs airborne) penalty
    fits = [cr.get_mountain_fitness() for cr in pop.creatures]
    links = [len(cr.get_expanded_links()) for cr in pop.creatures]

    peak_dists = [cr.get_best_peak_distance() for cr in pop.creatures]
    max_heights = [cr.get_max_height() for cr in pop.creatures]
    air_ratios = [cr.get_air_ratio() for cr in pop.creatures]
    supported_ratios = [cr.get_supported_ratio() for cr in pop.creatures]

    max_fit = float(np.max(fits))
    mean_fit = float(np.mean(fits))
    max_links = int(np.max(links))
    mean_links = float(np.mean(links))

    # Best creature (by fitness) for "best_*" metrics
    best_idx = int(np.argmax(fits))
    best_peak_dist = float(peak_dists[best_idx])
    mean_peak_dist = float(np.mean(peak_dists))

    best_max_height = float(max_heights[best_idx])
    mean_max_height = float(np.mean(max_heights))

    best_air_ratio = float(air_ratios[best_idx])
    mean_air_ratio = float(np.mean(air_ratios))

```

```
best_supported_ratio = float(supported_ratios[best_idx])
mean_supported_ratio = float(np.mean(supported_ratios))
```

```
dt = time.time() - t0
```

```
print(
    f"gen {gen:04d} | "
    f"max_fit={max_fit:.6f} mean_fit={mean_fit:.6f} | "
    f"best_peak_dist={best_peak_dist:.3f} mean_peak_dist={mean_peak_dist:.3f} | "
    f"best_height={best_max_height:.3f} mean_height={mean_max_height:.3f} | "
    f"best_air={best_air_ratio:.3f} mean_air={mean_air_ratio:.3f} | "
    f"max_links={max_links} mean_links={mean_links:.1f} | "
    f"time={dt:.2f}s"
)
```

```
writer.writerow([
    gen,
    max_fit,
    mean_fit,
    max_links,
    mean_links,
    best_peak_dist,
    mean_peak_dist,
    best_max_height,
    mean_max_height,
    best_air_ratio,
    mean_air_ratio,
    best_supported_ratio,
    mean_supported_ratio,
    dt
])
```

```
# Build fitness map for roulette selection
fit_map = population.Population.get_fitness_map(fits)
```

```
# Create next generation
new_creatures = []
for _ in range(len(pop.creatures)):
    p1_ind = _select_parent_index(fit_map, len(pop.creatures))
    p2_ind = _select_parent_index(fit_map, len(pop.creatures))
```

```
p1 = pop.creatures[p1_ind]
p2 = pop.creatures[p2_ind]
```

```
dna = genome.Genome.crossover(p1.dna, p2.dna)
dna = genome.Genome.point_mutate(dna, rate=point_rate, amount=point_amount)
dna = genome.Genome.shrink_mutate(dna, rate=shrink_rate)
dna = genome.Genome.grow_mutate(dna, rate=grow_rate)
```

```
cr_new = creature.Creature(gene_count)
cr_new.update_dna(dna)
```

```

    new_creatures.append(cr_new)

    # Elitism: copy best creature unchanged into index 0, and save DNA
    elite_src = pop.creatures[best_idx]

    elite_copy = creature.Creature(gene_count)
    elite_copy.update_dna(elite_src.dna)
    new_creatures[0] = elite_copy

    elite_path = os.path.join(elite_dir, f"elite_gen_{gen:04d}.csv")
    genome.Genome.to_csv(elite_src.dna, elite_path)

    pop.creatures = new_creatures

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--pop_size", type=int, default=10)
    parser.add_argument("--gene_count", type=int, default=3)
    parser.add_argument("--generations", type=int, default=50)
    parser.add_argument("--sim_steps", type=int, default=2400)

    parser.add_argument("--point_rate", type=float, default=0.1)
    parser.add_argument("--point_amount", type=float, default=0.25)
    parser.add_argument("--shrink_rate", type=float, default=0.25)
    parser.add_argument("--grow_rate", type=float, default=0.1)

    parser.add_argument("--seed", type=int, default=1)
    parser.add_argument("--out_csv", type=str, default="ga_baseline_log.csv")
    parser.add_argument("--elite_dir", type=str, default="elite_dna")

    args = parser.parse_args()

    run_ga(
        pop_size=args.pop_size,
        gene_count=args.gene_count,
        generations=args.generations,
        sim_steps=args.sim_steps,
        point_rate=args.point_rate,
        point_amount=args.point_amount,
        shrink_rate=args.shrink_rate,
        grow_rate=args.grow_rate,
        seed=args.seed,
        out_csv=args.out_csv,
        elite_dir=args.elite_dir,
    )

if __name__ == "__main__":
    main()

```


src/simulation.py — Unchanged (starter)

Yellow shading indicates differences relative to the starter code.

```
import pybullet as p
from multiprocessing import Pool

class Simulation:
    def __init__(self, sim_id=0):
        self.physicsClientId = p.connect(p.DIRECT)
        self.sim_id = sim_id

    def run_creature(self, cr, iterations=2400):
        pid = self.physicsClientId
        p.resetSimulation(physicsClientId=pid)
        p.setPhysicsEngineParameter(enableFileCaching=0, physicsClientId=pid)

        p.setGravity(0, 0, -10, physicsClientId=pid)
        plane_shape = p.createCollisionShape(p.GEOM_PLANE, physicsClientId=pid)
        floor = p.createMultiBody(plane_shape, plane_shape, physicsClientId=pid)

        xml_file = 'temp' + str(self.sim_id) + '.urdf'
        xml_str = cr.to_xml()
        with open(xml_file, 'w') as f:
            f.write(xml_str)

        cid = p.loadURDF(xml_file, physicsClientId=pid)

        p.resetBasePositionAndOrientation(cid, [0, 0, 2.5], [0, 0, 0, 1], physicsClientId=pid)

        for step in range(iterations):
            p.stepSimulation(physicsClientId=pid)
            if step % 24 == 0:
                self.update_motors(cid=cid, cr=cr)

            pos, orn = p.getBasePositionAndOrientation(cid, physicsClientId=pid)
            cr.update_position(pos)
            #print(pos[2])
            #print(cr.get_distance_travelled())

    def update_motors(self, cid, cr):
        """
        cid is the id in the physics engine
        cr is a creature object
        """
        for jid in range(p.getNumJoints(cid,
                                     physicsClientId=self.physicsClientId)):
            m = cr.get_motors()[jid]
```

```

p.setJointMotorControl2(cid, jid,
    controlMode=p.VELOCITY_CONTROL,
    targetVelocity=m.get_output(),
    force = 5,
    physicsClientId=self.physicsClientId)

```

You can add this to the Simulation class:

```

def eval_population(self, pop, iterations):
    for cr in pop.creatures:
        self.run_creature(cr, 2400)

```

```

class ThreadedSim():

```

```

    def __init__(self, pool_size):
        self.sims = [Simulation(i) for i in range(pool_size)]

```

```

    @staticmethod

```

```

    def static_run_creature(sim, cr, iterations):
        sim.run_creature(cr, iterations)
        return cr

```

```

    def eval_population(self, pop, iterations):

```

```

        """
        pop is a Population object
        iterations is frames in pybullet to run for at 240fps
        """

```

```

        pool_args = []
        start_ind = 0
        pool_size = len(self.sims)
        while start_ind < len(pop.creatures):
            this_pool_args = []
            for i in range(start_ind, start_ind + pool_size):
                if i == len(pop.creatures):# the end
                    break
                # work out the sim ind
                sim_ind = i % len(self.sims)
                this_pool_args.append([
                    self.sims[sim_ind],
                    pop.creatures[i],
                    iterations]
                )
            pool_args.append(this_pool_args)
            start_ind = start_ind + pool_size

```

```

        new_creatures = []
        for pool_argset in pool_args:
            with Pool(pool_size) as p:
                # it works on a copy of the creatures, so receive them
                creatures = p.starmap(ThreadedSim.static_run_creature, pool_argset)
                # and now put those creatures back into the main
                # self.creatures array

```

```
new_creatures.extend(creatures)
pop.creatures = new_creatures
```

src/simulation_mountain.py — New (not in starter)

Yellow shading indicates differences relative to the starter code.

```
"""simulation_mountain.py

Mountain-sandbox evaluation for the CM3020 Part B genetic algorithm task.

Key responsibilities:
- Build a bounded arena and load the mountain URDF.
- Spawn each creature near the arena edge (low height) to avoid "drop onto peak" artefacts.
- Track per-step statistics needed for fitness: peak distance, max height, and supported/airborne ratio.

This file is intended to support both:
- Fast, headless evaluation via PyBullet DIRECT mode (used during GA runs), and
- Deterministic replays of saved elites for evidence (video).
"""

import os
import pybullet as p
from multiprocessing import Pool
import time

class Simulation:
    def __init__(self, sim_id=0, arena_size=20, wall_height=1.0, gui=False):
        """
        Simulation wrapper for evaluating and (optionally) replaying creatures.

        Parameters
        -----
        sim_id : int
            Unique id used for temp URDF naming (useful if running multiple sims).
        arena_size : float
            Size of the bounded sandbox arena.
        wall_height : float
            Height of boundary walls to keep creatures within the arena.
        gui : bool
            If True, connect using PyBullet GUI for video/demo. If False, use DIRECT for fast GA evaluation.
        """
        self.gui = bool(gui)
        self.physicsClientId = p.connect(p.GUI if self.gui else p.DIRECT)

        self.sim_id = sim_id
        self.arena_size = arena_size
        self.wall_height = wall_height
        self.shapes_path = os.path.join(os.path.dirname(__file__), "shapes")

        # Temporary URDF output directory (keeps the repo clean and avoids path issues).
        self.tmp_urdf_dir = os.path.join(os.path.dirname(__file__), "tmp_urdf")
        os.makedirs(self.tmp_urdf_dir, exist_ok=True)
```

```

def _make_arena(self, pid):
    arena_size = float(self.arena_size)
    wall_height = float(self.wall_height)
    wall_thickness = 0.5

    # Floor
    floor_shape = p.createCollisionShape(
        shapeType=p.GEOM_BOX,
        halfExtents=[arena_size / 2, arena_size / 2, wall_thickness],
        physicsClientId=pid
    )
    floor_id = p.createMultiBody(
        baseMass=0,
        baseCollisionShapeIndex=floor_shape,
        basePosition=[0, 0, -wall_thickness],
        physicsClientId=pid
    )

    # Two walls along Y (north/south)
    wall_shape_y = p.createCollisionShape(
        shapeType=p.GEOM_BOX,
        halfExtents=[arena_size / 2, wall_thickness / 2, wall_height / 2],
        physicsClientId=pid
    )
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_shape_y,
        basePosition=[0, arena_size / 2, wall_height / 2], physicsClientId=pid)
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_shape_y,
        basePosition=[0, -arena_size / 2, wall_height / 2], physicsClientId=pid)

    # Two walls along X (east/west)
    wall_shape_x = p.createCollisionShape(
        shapeType=p.GEOM_BOX,
        halfExtents=[wall_thickness / 2, arena_size / 2, wall_height / 2],
        physicsClientId=pid
    )
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_shape_x,
        basePosition=[arena_size / 2, 0, wall_height / 2], physicsClientId=pid)
    p.createMultiBody(baseMass=0, baseCollisionShapeIndex=wall_shape_x,
        basePosition=[-arena_size / 2, 0, wall_height / 2], physicsClientId=pid)

    return floor_id

def _load_mountain(self, pid):
    p.setAdditionalSearchPath(self.shapes_path, physicsClientId=pid)
    mountain_position = (0, 0, -1)
    mountain_orientation = p.getQuaternionFromEuler((0, 0, 0))
    mountain_id = p.loadURDF(
        "gaussian_pyramid.urdf",
        mountain_position,
        mountain_orientation,
        useFixedBase=1,
        physicsClientId=pid
    )

```

```
)  
return mountain_id
```

```
def run_creature(self, cr, iterations=2400, motor_force=5.0, real_time=False, show_debug=False, real_time_step=1.0/240.0):  
    pid = self.physicsClientId  
    p.resetSimulation(physicsClientId=pid)  
    p.setPhysicsEngineParameter(enableFileCaching=0, physicsClientId=pid)  
    p.setGravity(0, 0, -10, physicsClientId=pid)
```

```
    floor_id = self._make_arena(pid)  
    mountain_id = self._load_mountain(pid)
```

```
    # Peak position for "closeness to the top" fitness:  
    # Use the mountain's AABB to get an accurate max Z in world coords.  
    aabb_min, aabb_max = p.getAABB(mountain_id, physicsClientId=pid)  
    peak_pos = (0.0, 0.0, float(aabb_max[2]))
```

```
    if self.gui:  
        p.resetDebugVisualizerCamera(  
            cameraDistance=18,  
            cameraYaw=45,  
            cameraPitch=-30,  
            cameraTargetPosition=[0, 0, 0],  
            physicsClientId=pid  
        )
```

```
    # Write creature URDF to a controlled temp directory.  
    xml_file = os.path.join(self.tmp_urdf_dir, f"temp_{self.sim_id}.urdf")  
    xml_str = cr.to_xml()  
    with open(xml_file, "w") as f:  
        f.write(xml_str)
```

```
    cid = p.loadURDF(xml_file, physicsClientId=pid)
```

```
    # The URDF is now loaded into the physics engine; remove the temp file to avoid clutter.  
    try:  
        os.remove(xml_file)  
    except OSError:  
        pass
```

```
    # Spawn near the edge of the arena at low height (prevents "cheating" by dropping onto the peak)  
    spawn_pos = [0, -self.arena_size / 2 + 2.0, 1.5]  
    p.resetBasePositionAndOrientation(cid, spawn_pos, [0, 0, 0, 1], physicsClientId=pid)
```

```
    for step in range(iterations):  
        p.stepSimulation(physicsClientId=pid)  
        if step % 24 == 0:  
            self.update_motors(cid=cid, cr=cr, motor_force=motor_force)
```

```
    pos, orn = p.getBasePositionAndOrientation(cid, physicsClientId=pid)
```

```
    # Supported if touching the floor OR the mountain (prevents "cheating" by flying)
```

```

supported = (
    len(p.getContactPoints(bodyA=cid, bodyB=floor_id, physicsClientId=pid)) > 0
    or len(p.getContactPoints(bodyA=cid, bodyB=mountain_id, physicsClientId=pid)) > 0
)

```

```

cr.update_position(pos, supported=supported, peak_pos=peak_pos)
if real_time and self.gui:
    time.sleep(real_time_step)

```

```

# Optional on-screen debug overlay for video/demo.
if show_debug and self.gui and (step % 120 == 0):
    # Use creature getters that your GA logger already uses.
    debug_lines = [
        f"step: {step}/{iterations}",
        f"best_peak_dist: {cr.get_best_peak_distance():.3f}",
        f"max_height: {cr.get_max_height():.3f}",
        f"air_ratio: {cr.get_air_ratio():.3f}",
        f"supported_ratio: {cr.get_supported_ratio():.3f}",
        f"fitness: {cr.get_mountain_fitness():.6f}",
    ]
    p.addUserDebugText(
        "\n".join(debug_lines),
        textPosition=[-8, -8, 6],
        textSize=1.2,
        lifeTime=0.2,
        physicsClientId=pid
    )

```

```

# Optional realtime pacing (useful for video recording).
if real_time and self.gui:
    time.sleep(real_time_step)

```

```

def update_motors(self, cid, cr, motor_force=5.0):
    pid = self.physicsClientId
    for jid in range(p.getNumJoints(cid, physicsClientId=pid)):
        m = cr.get_motors()[jid]
        p.setJointMotorControl2(
            cid,
            jid,
            controlMode=p.VELOCITY_CONTROL,
            targetVelocity=m.get_output(),
            force=float(motor_force),
            physicsClientId=pid
        )

```

```

def eval_population(self, pop, iterations):
    for cr in pop.creatures:
        self.run_creature(cr, iterations)

```

```

class ThreadedSim:

```

```
def __init__(self, pool_size, arena_size=20, wall_height=1.0):
    self.sims = [Simulation(i, arena_size=arena_size, wall_height=wall_height) for i in range(pool_size)]
```

```
@staticmethod
def static_run_creature(sim, cr, iterations):
    sim.run_creature(cr, iterations)
    return cr
```

```
def eval_population(self, pop, iterations):
    pool_args = []
    start_ind = 0
    pool_size = len(self.sims)
```

```
    while start_ind < len(pop.creatures):
        this_pool_args = []
        for i in range(start_ind, start_ind + pool_size):
            if i == len(pop.creatures):
                break
            sim_ind = i % len(self.sims)
            this_pool_args.append([self.sims[sim_ind], pop.creatures[i], iterations])
        pool_args.append(this_pool_args)
        start_ind += pool_size
```

```
    new_creatures = []
    for pool_argset in pool_args:
        with Pool(pool_size) as pool:
            creatures = pool.starmap(ThreadedSim.static_run_creature, pool_argset)
            new_creatures.extend(creatures)
    pop.creatures = new_creatures
```

src/starter.py — Modified

Yellow shading indicates differences relative to the starter code.

```
import os
import time
import pybullet as p
import pybullet_data as pd

def main():
    p.connect(p.GUI)
    p.setPhysicsEngineParameter(enableFileCaching=0)
    p.configureDebugVisualizer(p.COV_ENABLE_GUI, 0)

    p.setAdditionalSearchPath(pd.getDataPath())

    plane_shape = p.createCollisionShape(p.GEOM_PLANE)
    floor = p.createMultiBody(plane_shape, plane_shape)
    p.setGravity(0, 0, -10)

    # ---- ADD THIS: load the mountain ----
    mountain_urdf = os.path.join("src", "shapes", "gaussian_pyramid.urdf")
    p.loadURDF(mountain_urdf, (0, 0, -1), useFixedBase=1)
    # -----

    # Optional: set a sensible camera angle
    p.resetDebugVisualizerCamera(cameraDistance=12, cameraYaw=45, cameraPitch=-35, cameraTargetPosition=(0, 0, 2))

    try:
        while p.isConnected():
            p.stepSimulation()
            time.sleep(1.0 / 240.0)
    except KeyboardInterrupt:
        pass
    finally:
        if p.isConnected():
            p.disconnect()

if __name__ == "__main__":
    main()
```

src/test_creature.py — Unchanged (starter)

Yellow shading indicates differences relative to the starter code.

```
import unittest
import creature
import pybullet as p

class TestCreature(unittest.TestCase):
    def testCreatExists(self):
        self.assertIsNotNone(creature.Creature)

    def testCreatureGetFlatLinks(self):
        c = creature.Creature(gene_count=4)
        links = c.get_flat_links()
        self.assertEqual(len(links), 4)

    def testExpLinks(self):
        c = creature.Creature(gene_count=25)
        links = c.get_flat_links()
        exp_links = c.get_expanded_links()
        self.assertGreaterEqual(len(exp_links), len(links))

    def testToXMLNotNone(self):
        c = creature.Creature(gene_count=2)
        xml_str = c.to_xml()
        self.assertIsNotNone(xml_str)

    def testLoadXML(self):
        c = creature.Creature(gene_count=20)
        xml_str = c.to_xml()
        with open('test.urdf', 'w') as f:
            f.write(xml_str)
        p.connect(p.DIRECT)
        cid = p.loadURDF('test.urdf')
        self.assertIsNotNone(cid)

    def testMotor(self):
        m = creature.Motor(0.1, 0.5, 0.5)
        self.assertIsNotNone(m)

    def testMotorVal(self):
        m = creature.Motor(0.1, 0.5, 0.5)
        self.assertEqual(m.get_output(), 1)

    def testMotorVal2(self):
        m = creature.Motor(0.6, 0.5, 0.5)
        m.get_output()
        m.get_output()
        self.assertGreater(m.get_output(), 0)
```

```
def testDist(self):
    c = creature.Creature(3)
    c.update_position((0, 0, 0))
    d1 = c.get_distance_travelled()
    c.update_position((1, 1, 1))
    d2 = c.get_distance_travelled()
    self.assertGreater(d2, d1)
```

```
unittest.main()
```

src/test_ga.py — Unchanged (starter)

Yellow shading indicates differences relative to the starter code.

```
# If you on a Windows machine with any Python version
# or an M1 mac with any Python version
# or an Intel Mac with Python > 3.7
# this multi-threaded version does not work
# please use test_ga_single_thread.py on those setups

import unittest
import population
import simulation
import genome
import creature
import numpy as np

class TestGA(unittest.TestCase):
    def testBasicGA(self):
        pop = population.Population(pop_size=10,
                                     gene_count=3)
        sim = simulation.ThreadedSim(pool_size=1)
        #sim = simulation.Simulation()

        for iteration in range(1000):
            sim.eval_population(pop, 2400)
            fits = [cr.get_distance_travelled()
                    for cr in pop.creatures]
            links = [len(cr.get_expanded_links())
                    for cr in pop.creatures]
            print(iteration, "fittest:", np.round(np.max(fits), 3),
                  "mean:", np.round(np.mean(fits), 3), "mean links", np.round(np.mean(links)), "max links", np.round(np.max(links)))
            fit_map = population.Population.get_fitness_map(fits)
            new_creatures = []
            for i in range(len(pop.creatures)):
                p1_ind = population.Population.select_parent(fit_map)
                p2_ind = population.Population.select_parent(fit_map)
                p1 = pop.creatures[p1_ind]
                p2 = pop.creatures[p2_ind]
                # now we have the parents!
                dna = genome.Genome.crossover(p1.dna, p2.dna)
                dna = genome.Genome.point_mutate(dna, rate=0.1, amount=0.25)
                dna = genome.Genome.shrink_mutate(dna, rate=0.25)
                dna = genome.Genome.grow_mutate(dna, rate=0.1)
                cr = creature.Creature(1)
                cr.update_dna(dna)
                new_creatures.append(cr)
            # elitism
            max_fit = np.max(fits)
            for cr in pop.creatures:
                if cr.get_distance_travelled() == max_fit:
```

```
new_cr = creature.Creature(1)
new_cr.update_dna(cr.dna)
new_creatures[0] = new_cr
filename = "elite_"+str(iteration)+".csv"
genome.Genome.to_csv(cr.dna, filename)
break
```

```
pop.creatures = new_creatures
```

```
self.assertNotEqual(fits[0], 0)
```

```
unittest.main()
```

src/test_ga_no_threads.py — Unchanged (starter)

Yellow shading indicates differences relative to the starter code.

```
# If you on a Windows machine with any Python version
# or an M1 mac with any Python version
# or an Intel Mac with Python > 3.7
# the multi-threaded version does not work
# so instead, you can use this version.

import unittest
import population
import simulation
import genome
import creature
import numpy as np

class TestGA(unittest.TestCase):
    def testBasicGA(self):
        pop = population.Population(pop_size=10,
                                     gene_count=3)
        #sim = simulation.ThreadedSim(pool_size=1)
        sim = simulation.Simulation()

        for iteration in range(1000):
            # this is a non-threaded version
            # where we just call run_creature instead
            # of eval_population
            for cr in pop.creatures:
                sim.run_creature(cr, 2400)
            #sim.eval_population(pop, 2400)
            fits = [cr.get_distance_travelled()
                    for cr in pop.creatures]
            links = [len(cr.get_expanded_links())
                    for cr in pop.creatures]
            print(iteration, "fittest:", np.round(np.max(fits), 3),
                  "mean:", np.round(np.mean(fits), 3), "mean links", np.round(np.mean(links)), "max links", np.round(np.max(links)))
            fit_map = population.Population.get_fitness_map(fits)
            new_creatures = []
            for i in range(len(pop.creatures)):
                p1_ind = population.Population.select_parent(fit_map)
                p2_ind = population.Population.select_parent(fit_map)
                p1 = pop.creatures[p1_ind]
                p2 = pop.creatures[p2_ind]
                # now we have the parents!
                dna = genome.Genome.crossover(p1.dna, p2.dna)
                dna = genome.Genome.point_mutate(dna, rate=0.1, amount=0.25)
                dna = genome.Genome.shrink_mutate(dna, rate=0.25)
                dna = genome.Genome.grow_mutate(dna, rate=0.1)
                cr = creature.Creature(1)
                cr.update_dna(dna)
```

```
        new_creatures.append(cr)
    # elitism
    max_fit = np.max(fits)
    for cr in pop.creatures:
        if cr.get_distance_travelled() == max_fit:
            new_cr = creature.Creature(1)
            new_cr.update_dna(cr.dna)
            new_creatures[0] = new_cr
            filename = "elite_"+str(iteration)+".csv"
            genome.Genome.to_csv(cr.dna, filename)
            break

    pop.creatures = new_creatures

self.assertNotEqual(fits[0], 0)

unittest.main()
```

src/test_genome.py — Unchanged (starter)

Yellow shading indicates differences relative to the starter code.

```
import unittest
import genome
import numpy as np
import os

class GenomeTest (unittest.TestCase):
    def testClassExists(self):
        self.assertIsNotNone(genome.Genome)

    def testClassExists(self):
        self.assertIsNotNone(genome.Genome)

    def testRandomGene(self):
        self.assertIsNotNone(genome.Genome.get_random_gene)

    def testRandomGeneNotNone(self):
        self.assertIsNotNone(genome.Genome.get_random_gene(5))

    def testRandomGeneHasValues(self):
        gene = genome.Genome.get_random_gene(5)
        self.assertIsNotNone(gene[0])

    def testRandomGeneLength(self):
        gene = genome.Genome.get_random_gene(20)
        self.assertEqual(len(gene), 20)

    def testRandGenIsNumpyArrays(self):
        gene = genome.Genome.get_random_gene(20)
        self.assertEqual(type(gene), np.ndarray)

    def testRandomGenomeExists(self):
        data = genome.Genome.get_random_genome(20, 5)
        self.assertIsNotNone(data)

    def testGeneSpecExist(self):
        spec = genome.Genome.get_gene_spec()
        self.assertIsNotNone(spec)

    def testGeneSpecHasLinkLength(self):
        spec = genome.Genome.get_gene_spec()
        self.assertIsNotNone(spec['link-length'])

    def testGeneSpecHasLinkLength(self):
        spec = genome.Genome.get_gene_spec()
        self.assertIsNotNone(spec['link-length']['ind'])
```

```

def testGeneSpecScale(self):
    spec = genome.Genome.get_gene_spec()
    gene = genome.Genome.get_random_gene(20)
    self.assertGreater(gene[spec["link-length"]["ind"]], 0)

def testGeneToGeneDict(self):
    spec = genome.Genome.get_gene_spec()
    gene = genome.Genome.get_random_gene(len(spec))
    gene_dict = genome.Genome.get_gene_dict(gene, spec)
    self.assertIn("link-recurrence", gene_dict)

def testGenomeToDict(self):
    spec = genome.Genome.get_gene_spec()
    dna = genome.Genome.get_random_genome(len(spec), 3)
    genome_dicts = genome.Genome.get_genome_dicts(dna, spec)
    self.assertEqual(len(genome_dicts), 3)

def testFlatLinks(self):
    links = [
        genome.URDFLink(name="A", parent_name=None, recur=1),
        genome.URDFLink(name="B", parent_name="A", recur=2),
        genome.URDFLink(name="C", parent_name="B", recur=2)
    ]
    self.assertIsNotNone(links)

def testExpandLinks(self):
    links = [
        genome.URDFLink(name="A", parent_name="None", recur=1),
        genome.URDFLink(name="B", parent_name="A", recur=1),
        genome.URDFLink(name="C", parent_name="B", recur=2),
        genome.URDFLink(name="D", parent_name="C", recur=1),
    ]
    exp_links = [links[0]]
    genome.Genome.expandLinks(links[0], links[0].name, links, exp_links)
    self.assertEqual(len(exp_links), 6)

def testCrossover(self):
    g1 = [[1], [2], [3]]
    g2 = [[4], [5], [6]]
    for i in range(10):
        g3 = genome.Genome.crossover(g1, g2)
        self.assertGreater(len(g3), 0)

def test_point(self):
    g1 = np.array([[1.0], [2.0], [3.0]])
    g2 = genome.Genome.point_mutate(g1, rate=1, amount=0.25)
    self.assertFalse(np.array_equal(g1, g2))

def test_point_range(self):
    g1 = np.array([[1.0], [0.0], [1.0], [0.0]])

```

```

for i in range(100):
    g2 = genome.Genome.point_mutate(g1, rate=1, amount=0.25)
    self.assertLessEqual(np.max(g2), 1.0)
    self.assertGreaterEqual(np.min(g2), 0.0)

def test_shrink(self):
    g1 = np.array([[1.0], [2.0]])
    g2 = genome.Genome.shrink_mutate(g1, rate=1.0)
    # should def. shrink as rate = 1
    self.assertEqual(len(g2), 1)

def test_shrink2(self):
    g1 = np.array([[1.0], [2.0]])
    g2 = genome.Genome.shrink_mutate(g1, rate=0.0)
    # should not shrink as rate = 0
    self.assertEqual(len(g2), 2)

def test_shrink3(self):
    g1 = np.array([[1.0]])
    g2 = genome.Genome.shrink_mutate(g1, rate=1.0)
    # should not shrink if already len 1
    self.assertEqual(len(g2), 1)

def test_grow1(self):
    g1 = np.array([[1.0], [2.0]])
    g2 = genome.Genome.grow_mutate(g1, rate=1)
    self.assertGreater(len(g2), len(g1))

def test_grow2(self):
    g1 = np.array([[1.0], [2.0]])
    g2 = genome.Genome.grow_mutate(g1, rate=0)
    self.assertEqual(len(g2), len(g1))

def test_tocsv(self):
    g1 = [[1,2,3]]
    genome.Genome.to_csv(g1, 'test.csv')
    self.assertTrue(os.path.exists('test.csv'))

def test_tocsv_content(self):
    g1 = [[1,2,3]]
    genome.Genome.to_csv(g1, 'test.csv')
    expect = "1,2,3,\n"
    with open('test.csv') as f:
        csv_str = f.read()
    self.assertEqual(csv_str, expect)

def test_tocsv_content2(self):
    g1 = [[1,2,3], [4,5,6]]
    genome.Genome.to_csv(g1, 'test.csv')
    expect = "1,2,3,\n4,5,6,\n"
    with open('test.csv') as f:
        csv_str = f.read()

```

```
self.assertEqual(csv_str, expect)

def test_from_csv(self):
    g1 = [[1,2,3]]
    genome.Genome.to_csv(g1, 'test.csv')
    g2 = genome.Genome.from_csv('test.csv')
    print(g1, g2)
    self.assertTrue(np.array_equal(g1, g2))

def test_from_csv2(self):
    g1 = [[1,2,3], [4,5,6]]
    genome.Genome.to_csv(g1, 'test.csv')
    g2 = genome.Genome.from_csv('test.csv')
    print(g1, g2)
    self.assertTrue(np.array_equal(g1, g2))

unittest.main()
```

src/test_population.py — Unchanged (starter)

Yellow shading indicates differences relative to the starter code.

```
import unittest
import population
import numpy as np

class TestPop(unittest.TestCase):
    ## check for a parent id in the range 0-2
    def testSelPar(self):
        fits = [2.5, 1.2, 3.4]
        fitmap = population.Population.get_fitness_map(fits)
        pid = population.Population.select_parent(fitmap)
        self.assertLess(pid, 3)

    ## parent id should be 1 as the first fitness is zero
    ## second is 1000 and third is 0.1 , so second should
    ## almost always be selected
    def testSelPar2(self):
        fits = [0, 1000, 0.1]
        fitmap = population.Population.get_fitness_map(fits)
        pid = population.Population.select_parent(fitmap)
        self.assertEqual(pid, 1)

unittest.main()
```

src/test_simulation.py — Unchanged (starter)

Yellow shading indicates differences relative to the starter code.

```
import unittest
import simulation
import creature
import population

class TestSim(unittest.TestCase):
    def testSimExists(self):
        sim = simulation.Simulation()
        self.assertIsNotNone(sim)

    def testSimId(self):
        sim = simulation.Simulation()
        self.assertIsNotNone(sim.physicsClientId)

    def testRun(self):
        sim = simulation.Simulation()
        self.assertIsNotNone(sim.run_creature)

    def testPos(self):
        sim = simulation.Simulation()
        cr = creature.Creature(gene_count = 3)
        sim.run_creature(cr)
        self.assertNotEqual(cr.start_position, cr.last_position)

    def testPop(self):
        pop = population.Population(pop_size=5, gene_count=3)
        sim = simulation.Simulation()
        for cr in pop.creatures:
            sim.run_creature(cr)
        dists = [cr.get_distance_travelled() for cr in pop.creatures]
        print(dists)
        self.assertIsNotNone(dists)

## uncomment this to test the
## multi-threaded sim
# def testProc(self):
#     pop = population.Population(pop_size=20, gene_count=3)
#     tsim = simulation.ThreadedSim(pool_size=8)
#     tsim.eval_population(pop, 2400)
#     dists = [cr.get_distance_travelled() for cr in pop.creatures]
#     print(dists)
#     self.assertIsNotNone(dists)

def testProcNoThread(self):
    pop = population.Population(pop_size=20, gene_count=3)
    sim = simulation.Simulation()
    sim.eval_population(pop, 2400)
```

```
dists = [cr.get_distance_travelled() for cr in pop.creatures]
print(dists)
self.assertIsNotNone(dists)
```

```
unittest.main()
```